Python 멀티스레딩 · 멀티프로세스

Threading · Multiprocessing · 로봇 멀티태스킹

cchyun@gmail.com

Part 1. 멀티태스킹 기초 개념

왜 멀티태스킹이 필요한가?

- **로봇은 여러 작업을 동시에 처리해야 하는 복잡한 시스템**

  - **동시 센서 읽기 시나리오**
    - 이동 중 초음파 센서로 거리 측정
    - 카메라로 장애물 인식
    - 동시에 바퀴 모터에 명령 전송

  - **문제점: Single Thread 방식**
    - 카메라 영상 처리 중(약 100~200ms) 초음파 센서 값을 읽지 못함
    - 이 짧은 지연 시간 동안 로봇이 장애물을 발견하지 못하고 충돌 가능

  - **해결책: 멀티태스킹**
    - **I/O 바운드 작업** 처리 시 응답을 기다리는 동안 다른 작업 수행
    - 시스템의 **응답성** 극대화

프로세스(Process) vs 스레드(Thread)

- **비유: "주방의 요리사"로 이해하기**

  - **프로세스 (Process)**
    - 실행 중인 프로그램의 **독립적인 인스턴스**
    - **독립된 메모리 공간**과 Python 인터프리터 보유
    - 비유: 별도의 주방과 도구를 가진 요리사
    - 한 요리사가 도구를 망가뜨려도 다른 주방에 영향 없음
    - 데이터 전달 시 `Queue` 같은 채널 필요, 생성 자원 소모 큼

  - **스레드 (Thread)**
    - 하나의 프로세스 내에서 실행되는 **여러 흐름**
    - 같은 프로세스 내의 **메모리 공간 공유**
    - 비유: 한 주방에서 같은 조리 도구·재료를 공유하며 작업하는 여러 요리사
    - 데이터 공유가 쉬운 반면, 동시 접근 시 문제 발생 가능 → 관리 필요

Python GIL (Global Interpreter Lock)

- **GIL: Python의 독특한 특징**
  - **GIL이란?**
    - 한 번에 하나의 네이티브 스레드만 Python 바이트코드를 실행하도록 제한하는 '잠금장치'
  - **CPU 집약적 작업 (이미지 처리, 경로 계획 등)**
    - 멀티스레딩으로 돌려도 실제 속도가 빨라지지 않음
    - → `multiprocessing` 모듈 사용 필요
  - **I/O 바운드 작업 (센서 통신, 네트워크)**
    - CPU를 거의 쓰지 않고 대기 시간이 많음
    - GIL은 대기 시간에 다른 스레드에게 실행권을 넘김
    - → **로봇의 여러 센서를 동시에 관리하는 데 멀티스레딩이 매우 효율적**

멀티스레딩 vs 멀티프로세싱 선택 기준

구분	멀티스레딩	멀티프로세싱
메모리	프로세스 내 공유	각 프로세스 독립
GIL 영향	받음	받지 않음
적합한 작업	I/O 바운드	CPU 바운드
생성 비용	낮음	높음
데이터 공유	쉬움 (Lock 필요)	Queue/Pipe 필요
사용 예시	시리얼 포트 읽기, 네트워크 요청, 하트비트	이미지 처리, AI 추론, 경로 계획

Part 2. 멀티스레딩 (threading 모듈)

Thread 생성 · start() · join()

- **스레드를 만들고 실행하는 기본 흐름**

```python
import threading
import time

def sensor_reader(name):
    print(f"{name} 읽기 시작")
    time.sleep(2)  # 센서 응답 대기 시뮬레이션
    print(f"{name} 읽기 완료")

# 1. Thread 생성 (target=실행할 함수, args=인수)
t = threading.Thread(target=sensor_reader, args=("Lidar",))

# 2. 스레드 시작
t.start()

# 3. 스레드가 끝날 때까지 메인 프로그램 대기
t.join()
print("모든 센서 작업 완료")
```

Daemon(데몬) 스레드

- **메인 프로그램 종료 시 함께 강제 종료되는 스레드**
  - **용도**
    - 로봇의 상태를 주기적으로 로깅
    - 백그라운드에서 하트비트(Heartbeat) 신호 전송
    - 메인 프로그램이 꺼졌는데 로깅 스레드가 살아있으면 프로그램이 종료 안 됨
  - **설정 방법**

```python
import threading

def heartbeat():
    while True:
        print("Robot alive...")
        time.sleep(0.1)

# daemon=True 로 생성 시 설정
t = threading.Thread(target=heartbeat, daemon=True)
t.start()
# 메인 프로그램 종료 시 데몬 스레드도 자동 종료
```

Lock을 이용한 공유 자원 보호

- **Race Condition (경쟁 상태)**: 여러 스레드가 하나의 변수·하드웨어 포트에 동시 접근 시 데이터가 꼬이는 문제
  - **해결책: Lock(자물쇠) 사용**

```
import threading

lock = threading.Lock()
telemetry_data = 0

def update_data():
    global telemetry_data
    with lock:  # 자물쇠 잠금 (작업 후 자동으로 풀림)
        # 안전하게 공유 자원 수정
        telemetry_data += 1
```

- `with lock:` 블록 안에서만 공유 자원 접근
- 한 스레드가 작업 중이면 다른 스레드는 대기

스레드 안전(Thread-safe) 코드 작성 방법

- **3가지 원칙**

1. **Lock 사용**
   - 공유 변수에 접근할 때 반드시 `threading.Lock()` 사용
   - 한 번에 하나의 스레드만 접근하도록 제어

2. **Queue 활용**
   - `queue.Queue` 모듈: 내부적으로 Lock이 구현되어 있음
   - 스레드 간에 안전하게 데이터를 주고받을 수 있음

3. **공유 상태 최소화**
   - 가급적 스레드 간에 데이터를 공유하지 않도록 설계
   - 독립적인 결과물만 내놓는 구조가 가장 안전

입문자가 자주 하는 실수 (멀티스레딩)

- **경쟁 상태 (Race Condition)**
  - 두 개 이상의 스레드가 하나의 데이터를 동시에 수정
  - 실행 타이밍에 따라 결과가 매번 달라지는 현상
  - **반드시 Lock을 사용해야 함**

- **데드락 (Deadlock)**
  - 두 스레드가 서로 상대방이 가진 Lock이 풀리기만을 기다리며 무한히 멈춤
  - 예시
    - A스레드: 자물쇠1 보유, 자물쇠2 대기
    - B스레드: 자물쇠2 보유, 자물쇠1 대기
    - → 서로 무한 대기 상태 발생

(종합 예제) 여러 센서를 동시에 읽기

```python
import threading, time, random

robot_telemetry = {"lidar": 0, "ultrasonic": 0}
data_lock = threading.Lock()

def read_lidar():
    while True:
        distance = random.uniform(0.1, 10.0)
        with data_lock:
            robot_telemetry["lidar"] = distance
        time.sleep(0.1)  # 10Hz

def read_ultrasonic():
    while True:
        time.sleep(0.5)
        distance = random.uniform(20, 400)
        with data_lock:
            robot_telemetry["ultrasonic"] = distance
```

(종합 예제) 메인 제어 루프

```python
if __name__ == "__main__":
    # 데몬 스레드: 메인 종료 시 함께 종료
    t1 = threading.Thread(target=read_lidar, daemon=True)
    t2 = threading.Thread(target=read_ultrasonic, daemon=True)

    t1.start()
    t2.start()

    try:
        while True:
            with data_lock:
                print(f"[Main Control] Lidar: {robot_telemetry['lidar']:.2f}m, "
                      f"Ultrasonic: {robot_telemetry['ultrasonic']:.2f}cm")
            time.sleep(1.0)
    except KeyboardInterrupt:
        print("\n로봇 제어 프로그램을 종료합니다.")
```

실습 팁: Lock을 제거한 후 실행해 보고 어떤 문제가 생기는지 직접 확인해 보세요

Part 3. 멀티프로세싱

(multiprocessing 모듈)

multiprocessing 기본 사용법

- **Process 생성 · start() · join()**
  - `Process` 객체 생성 시 실행할 함수(`target`)와 인자(`args`)를 지정
  - `start()`: 프로세스 시작
  - `join()`: 해당 프로세스가 종료될 때까지 부모 프로세스 대기

- **Queue를 이용한 프로세스 간 통신**
  - 프로세스는 메모리를 공유하지 않으므로 `Queue`("우체통")로 데이터 교환
  - `put()`: 데이터 전송 / `get()`: 데이터 수신

- **Event를 이용한 프로세스 간 신호**
  - 모든 프로세스가 공유하는 "정지 버튼"
  - `set()`: 신호 발송 / `is_set()`: 신호 확인
  - 비상 정지, 일괄 종료에 활용

로봇 개발에서의 멀티프로세싱 패턴

- **"3-프로세스 자동화 패턴"**
  - **센서/카메라 프로세스**
    - 최대한 빠른 속도(예: 60 FPS)로 데이터를 읽어 Queue에 전송
  - **추론/분석 프로세스**
    - AI 모델(CPU/GPU 집약적)로 데이터 분석
    - 분석 속도가 느려도(예: 10 FPS) 다른 프로세스를 방해하지 않음
  - **제어 프로세스**
    - 로봇의 물리적 움직임 관리
    - AI 추론이 늦어지더라도 독립적으로 동작
    - 일정 주기(Heartbeat)로 하드웨어와 통신하여 시스템 안정성 유지

Pool을 이용한 병렬 처리

- **동일한 작업을 여러 입력값에 대해 병렬 실행 (데이터 병렬 처리)**
  - `Pool(processes=N)` : N개의 워커 프로세스 풀 생성
  - `map(함수, 데이터_리스트)` : 데이터를 분배하여 처리 후 결과 리스트 반환

```python
from multiprocessing import Pool

def process_sensor_data(data):
    # 복잡한 필터링·계산 수행
    return data * 2

if __name__ == "__main__":
    sensor_readings = [1.2, 3.4, 5.6, 7.8, 9.0]

    with Pool(processes=4) as pool:
        results = pool.map(process_sensor_data, sensor_readings)
    print(results)  # [2.4, 6.8, 11.2, 15.6, 18.0]
```

대량의 센서 데이터 필터링, 복잡한 수치 계산에 효율적

입문자가 자주 하는 실수 (멀티프로세싱)

- `if __name__ == '__main__':` **누락**
  - 윈도우·macOS에서 이 조건문 없이 프로세스 생성 시
  - → 프로세스가 무한히 복제되는 런타임 오류 발생

- **변수 직접 공유 시도**
  - 프로세스는 독립된 메모리를 가짐
  - 전역 변수를 수정해도 다른 프로세스에는 반영되지 않음
  - → 반드시 `Queue`나 `Pipe` 사용

- **무분별한** `terminate()` **사용**
  - `terminate()`로 프로세스를 강제 종료하면
  - → 해당 프로세스가 보유한 Lock·Queue가 깨져 전체 시스템 **데드락** 위험

(예제) AI 추론과 센서 읽기 분리 (1/2)

```python
import multiprocessing, time, random

# 1. 센서 데이터를 읽는 워커 프로세스
def sensor_worker(sensor_queue, stop_event):
    print("센서 프로세스 시작")
    while not stop_event.is_set():
        data = random.uniform(0, 100)
        sensor_queue.put(data)
        time.sleep(0.01)  # 100Hz 속도로 읽기
    print("센서 프로세스 종료")

# 2. AI 추론을 수행하는 워커 프로세스
def ai_inference_worker(sensor_queue, stop_event):
    print("AI 추론 프로세스 시작")
    while not stop_event.is_set():
        try:
            data = sensor_queue.get(timeout=1)
            print(f"[AI] 데이터 {data:.2f} 분석 중...")
            time.sleep(0.2)  # 무거운 AI 연산 시뮬레이션
            print(f"[AI] 결과: 장애물 없음")
        except:
            continue
    print("AI 추론 프로세스 종료")
```

(예제) AI 추론과 센서 읽기 분리 (2/2)

```python
if __name__ == '__main__':
    # 프로세스 간 통신: Queue, 신호 전달: Event
    sensor_q = multiprocessing.Queue()
    stop_sig = multiprocessing.Event()

    # 프로세스 객체 생성
    p1 = multiprocessing.Process(
            target=sensor_worker, args=(sensor_q, stop_sig))
    p2 = multiprocessing.Process(
            target=ai_inference_worker, args=(sensor_q, stop_sig))

    p1.start()
    p2.start()

    try:
        time.sleep(5)  # 5초 동안 실행
    except KeyboardInterrupt:
        pass
    finally:
        stop_sig.set()   # 종료 신호 전송
        p1.join()
        p2.join()
        print("모든 시스템이 안전하게 종료되었습니다.")
```

실습 체크리스트

- [ ] **진입점 보호**
  - 코드가 `if __name__ == "__main__":` 블록 내부에서 시작되도록 작성했는가?
- [ ] **작업 유형 분류**
  - 구현하려는 작업이 **CPU 집약적**(계산 위주)인지 **I/O 집약적**(대기 위주)인지 구분했는가?
- [ ] **데이터 통신 설계**
  - 멀티프로세싱 사용 시 `Queue`나 `Pipe`를 적절히 설정했는가?
- [ ] **프로세스/스레드 종료**
  - 작업이 끝난 후 `join()`을 호출하여 모든 실행 단위가 정상 종료되도록 관리했는가?
- [ ] **예외 처리**
  - 하드웨어 연결 끊김 등에 대비해 `try-except` 블록으로 예외 처리를 수행했는가?